AI VIET NAM
@aivietnam.edu.vn

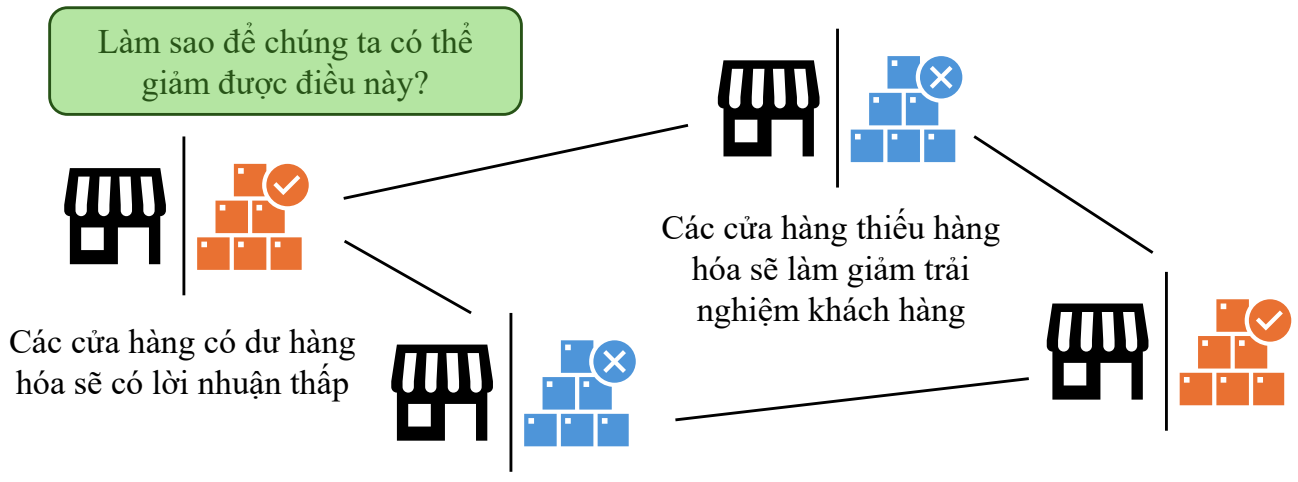
AI VIET NAM – AI COURSE 2025

Project: Xây dựng công cụ tối ưu hoá quy trình phân bổ hàng hoá cho chuỗi cửa hàng

Dr. Nguyễn Thái Hà và Nguyễn Phúc Thịnh và Nguyễn Thọ Anh Khoa

I. Giới thiệu

Trong bối cảnh thị trường bán lẻ cạnh tranh khốc liệt, việc quản lý hàng tồn kho hiệu quả là một trong những yếu tố then chốt quyết định sự thành bại của một doanh nghiệp. Các chuỗi cửa hàng lớn, với mạng lưới hàng trăm chi nhánh, phải đối mặt với bài toán điều phối hàng hóa cực kỳ phức tạp. Sự khác biệt về vị trí địa lý, đặc điểm khách hàng và nhu cầu thị trường tại mỗi điểm bán dẫn đến một thách thức chung: **mất cân bằng tồn kho (inventory imbalance)**. Tình trạng này gây ra hệ quả kép: một số cửa hàng thì thừa hàng, làm phát sinh chi phí lưu kho và đọng vốn; trong khi các cửa hàng khác lại thiếu hàng, dẫn đến mất doanh thu và suy giảm sự hài lòng của khách hàng.



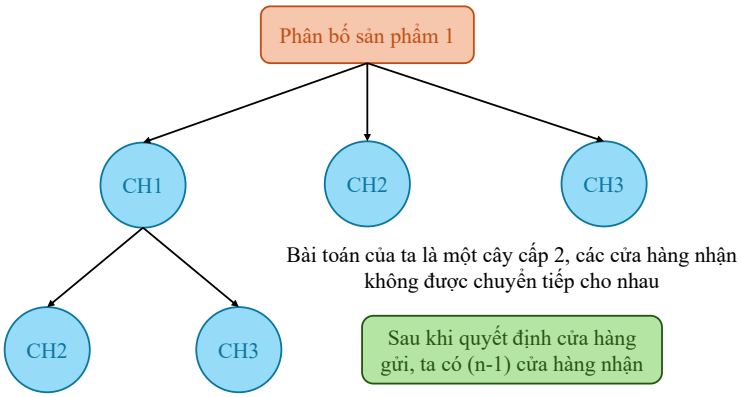
Hình 1: Bối cảnh thực tế của một chuỗi cửa hàng bán lẻ với thách thức về mất cân bằng tồn kho và nhu cầu về một giải pháp chuyển hàng thông minh và nhanh chóng.

Việc điều phối hàng hóa một cách thủ công, dựa nhiều vào kinh nghiệm cá nhân, thường không đủ nhanh nhạy và chính xác để giải quyết triệt để vấn đề. Hình bên phải minh họa sự bùng nổ của các lựa chọn: chỉ với 3 cửa hàng, người quản lý đã phải xem xét $3 \times 2 = 6$ tuyến điều chuyển hàng khác nhau. Nếu áp dụng vào bài toán thực tế với 300 cửa hàng, con số này sẽ là $300 \times 299 = 89,700$ lựa chọn. Việc đánh giá thủ công một không gian quyết định khổng lồ như vậy là bất khả thi.

	CH1	CH2	CH3
CH1		10	20
CH2	0		0
CH3	0	0	

Hình 2: Sự bùng nổ tổ hợp của các quyết định điều chuyển. Với n cửa hàng, một người quản lý phải cân nhắc $n(n-1)$ luồng vận chuyển hàng hóa. Với ví dụ $n = 300$, có gần 90,000 quyết định cần được đưa ra.

Do đó, nhu cầu về một hệ thống thông minh, có khả năng tự động hóa và tối ưu hóa các quyết định phân bổ hàng hóa trở nên cấp thiết hơn bao giờ hết. Chuyên đề này sẽ hướng dẫn học viên xây dựng một hệ thống tối ưu hóa phân bổ hàng hóa (Inventory Transfer Optimization System), ứng dụng nội dung bài học của khoa học dữ liệu và giải thuật di truyền (Genetic Algorithm) để giải quyết bài toán thực tiễn này.



Hình 3: Mô hình hóa bài toán phân bổ dưới dạng cây quyết định 2 cấp. Trong thực tế chúng ta có thể góp chuyển chở hàng.

Để hệ thống hóa không gian quyết định này, chúng ta có thể mô hình hóa bài toán dưới dạng một cây quyết định 2 cấp, như được minh họa trong Hình bên. Cách tiếp cận này giúp làm rõ quy trình ra quyết định:

- **Cấp 1 (Chọn cửa hàng gửi):** Đầu tiên, hệ thống phải quyết định sẽ lấy sản phẩm từ cửa hàng nào trong tổng số n cửa hàng.
- **Cấp 2 (Chọn cửa hàng nhận):** Sau khi đã xác định được cửa hàng gửi, hệ thống sẽ tiếp tục chọn một cửa hàng nhận từ $n-1$ cửa hàng còn lại.

Một giả định quan trọng được thể hiện trong mô hình này là các cửa hàng nhận sẽ không thực hiện chuyển tiếp hàng hóa đi nơi khác. Đây là một bài toán điều chuyển một lượt **(single-hop)**, giúp đơn giản hóa vấn đề nhưng vẫn giữ được tính thực tiễn cốt lõi. Cách cấu trúc này là tiền đề để chúng ta xây dựng một mô hình tối ưu hóa chính xác.

Mục tiêu bài viết: Chuyên đề này được thiết kế nhằm giúp học viên hiểu rõ cách ứng dụng khoa học dữ liệu vào một bài toán tối ưu thực tế, thay vì tập trung vào việc xây dựng một hệ thống tối ưu phức tạp. Người học sẽ được tiếp cận hai hướng giải quyết khác nhau là: rule-based

và genetic algorithm. Qua đó rèn luyện tư duy phân tích, kỹ năng mô hình hóa và khả năng đánh giá trade-offs giữa các thuật toán.

Nội dung được xây dựng phù hợp cho:

- **Sinh viên ngành AI/Data Science:** Học cách áp dụng optimization vào bài toán thực tế và thực hành trực tiếp với Pandas/NumPy.
- **Người trái ngành muốn chuyển sang Data/AI:** Làm một dự án thực hành có tính ứng dụng cao, dễ dàng trình bày trong portfolio.
- **Tư vấn viên mới vào nghề (Beginner Consultants):** Hiểu và áp dụng framework tư duy tối ưu, nắm được cách cân nhắc giữa các giải pháp kỹ thuật.

Bài học tập trung vào ý tưởng và phương pháp tiếp cận, mang lại nền tảng vững chắc để người học mở rộng sang các bài toán lớn hơn trong tương lai, đồng thời đảm bảo tuân thủ tính bảo mật của dự án đã triển khai trong thực tế.

Danh sách các module trong project. Để tiện theo dõi, bảng dưới đây tổng hợp các phần chính trong module của chúng ta. Toàn bộ dự án được chứa trong link Github đây.

Bảng Thành phần Project

Thành phần	Ý nghĩa và Chức năng
`src/main.py`	File thực thi chính của dự án. Chịu trách nhiệm điều phối toàn bộ quy trình: khởi tạo, tạo dữ liệu (nếu cần), chạy phân tích, thực thi các thuật toán tối ưu và lưu kết quả.
`src/config.py`	Tệp cấu hình trung tâm, chứa tất cả các hằng số và tham số quan trọng như đường dẫn thư mục, ngưỡng tồn kho an toàn (`MIN_INVENTORY_DAYS`, `MAX_INVENTORY_DAYS`), và các tham số cho thuật toán di truyền (kích thước quần thể, tỷ lệ lai ghép, đột biến).
`src/engine/analyzer.py`	Chứa lớp `InventoryAnalyzer`. Module này thực hiện các phân tích: tính toán doanh số trung bình hàng ngày, xác định mức tồn kho an toàn, và quan trọng nhất là xác định các sản phẩm đang **thừa** (*excess*) và **thiếu** (*needed*) tại mỗi cửa hàng cho các bước tiếp theo.
`src/engine/rule_based.py`	Triển khai thuật toán tối ưu dựa trên luật (`RuleBasedOptimizer`). Đây là một phương pháp tiếp cận tham lam (greedy), ưu tiên đáp ứng các nhu cầu từ nguồn cung có chi phí vận chuyển thấp nhất.
`src/engine/genetic_algorithm`	Triển khai thuật toán di truyền (`GeneticAlgorithmOptimizer`). Đây là phương pháp tối ưu hóa phức tạp hơn, tìm kiếm giải pháp tối ưu toàn cục bằng cách mô phỏng quá trình tiến hóa tự nhiên (chọn lọc, lai ghép, đột biến).
`src/engine/results_manager.p`	Quản lý việc lưu trữ và định dạng các kết quả đầu ra, bao gồm kế hoạch điều chuyển hàng hóa và các bản tóm tắt hiệu suất.
`data/`	Thư mục chứa tất cả các tệp dữ liệu đầu vào ở định dạng CSV, bao gồm dữ liệu bán hàng, tồn kho, thông tin sản phẩm, cửa hàng và ma trận chi phí vận chuyển.
`notebooks/`	Jupyter Notebook mẫu của dự án này với phân tích chi tiết, trực quan và từng bước của toàn bộ quy trình.

Mục lục

I. Giới thiệu **1**

II. Lý thuyết **6**

II.1. Tại sao cần tối ưu hóa phân bổ hàng hóa? 6

II.2. Định nghĩa bài toán 7

III. Xây dựng hệ thống **9**

III.1. Đọc dữ liệu 10

III.2. Phương pháp dựa trên các luật 12

III.3. Phương pháp thuật toán di truyền 15

III.4. So sánh hiệu năng và lựa chọn thuật toán 19

IV. Hướng cải tiến **20**

IV.1. Tích hợp dữ liệu thực và mô hình dự báo 20

IV.2. Cải tiến và tối ưu hóa giải thuật di truyền 21

IV.3. Áp dụng thuật toán đa mục tiêu (NSGA-II) 22

V. Câu hỏi trắc nghiệm **24**

VI. Tài liệu tham khảo **26**

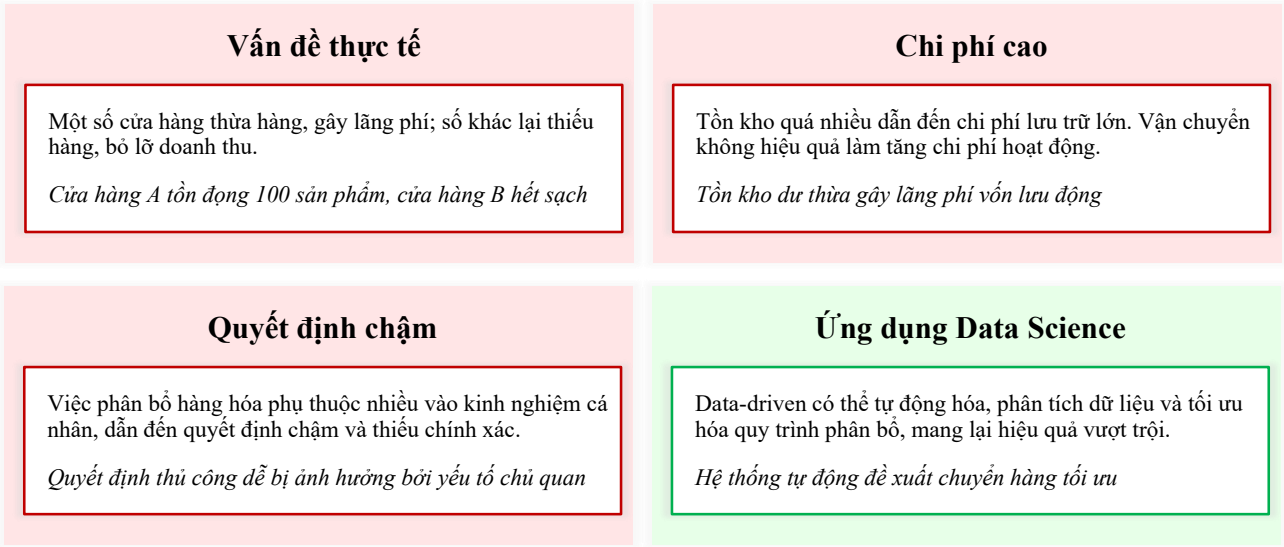
Phụ lục **26**

II. Lý thuyết

II.1. Tại sao cần tối ưu hóa phân bổ hàng hóa?

Trước khi đi vào giải pháp kỹ thuật, chúng ta cần hiểu rõ các vấn đề mà một doanh nghiệp phải đối mặt khi hệ thống phân bổ hàng hóa không hiệu quả. Như được tóm tắt trong Hình 4, có bốn động lực chính thúc đẩy nhu cầu cấp thiết này.

Quay trở lại vấn đề, như được tóm tắt trong Hình 4, có bốn động lực chính thúc đẩy nhu cầu cấp thiết về tối ưu hóa phân bổ hàng hóa.



Hình 4: Tóm tắt và ví dụ ba vấn đề và một hướng giải quyết cho bài toán tối ưu.

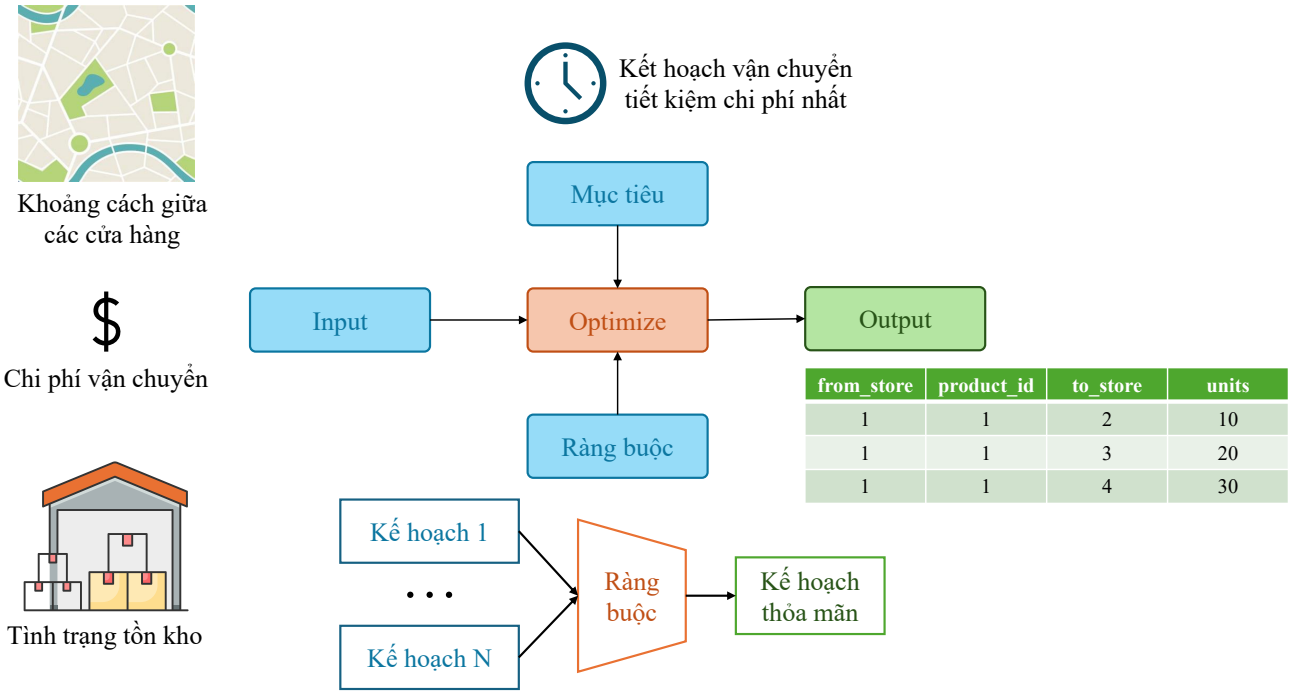
Mỗi khía cạnh này đều ẩn chứa những thách thức và cơ hội riêng:

- Về mặt thực tế, sự mất cân bằng tồn kho là một lãng phí tài nguyên hữu hình. Ví dụ về "Cửa hàng A tồn kho 100 sản phẩm, cửa hàng B hết sạch" không chỉ đơn thuần là vấn đề lưu trữ, mà nó trực tiếp dẫn đến việc bỏ lỡ doanh thu tại cửa hàng B và làm suy giảm trải nghiệm của khách hàng.
- Về chi phí, bài toán không chỉ dừng lại ở chi phí vận chuyển. Tồn kho dư thừa gây lãng phí vốn lưu động, phát sinh chi phí bảo quản, bảo hiểm, và đối mặt với rủi ro hàng hóa lỗi thời hoặc hư hỏng. Một hệ thống không tối ưu sẽ khuếch đại tất cả các loại chi phí này.
- Về quyết định, sự phụ thuộc vào kinh nghiệm cá nhân khiến quy trình trở nên chậm chạp và thiếu nhất quán. Các quyết định thủ công khó có thể phản ứng kịp thời trước các biến động đột ngột của thị trường (ví dụ: một xu hướng mới, một chương trình khuyến mãi của đối thủ) và dễ bị sai lệch bởi yếu tố chủ quan.
- Về cơ hội, đây chính là khía cạnh then chốt. Dữ liệu bán hàng, tồn kho, và lưu lượng khách hàng không còn là những con số. Khi được khai thác đúng cách, chúng trở thành nền tảng

cho một hệ thống **data-driven** có khả năng tự động hóa toàn bộ quy trình: từ dự báo nhu cầu, phân tích các kịch bản điều chuyển, cho đến việc đề xuất lộ trình tối ưu nhất. Đây chính là giải pháp mà chúng ta sẽ xây dựng.

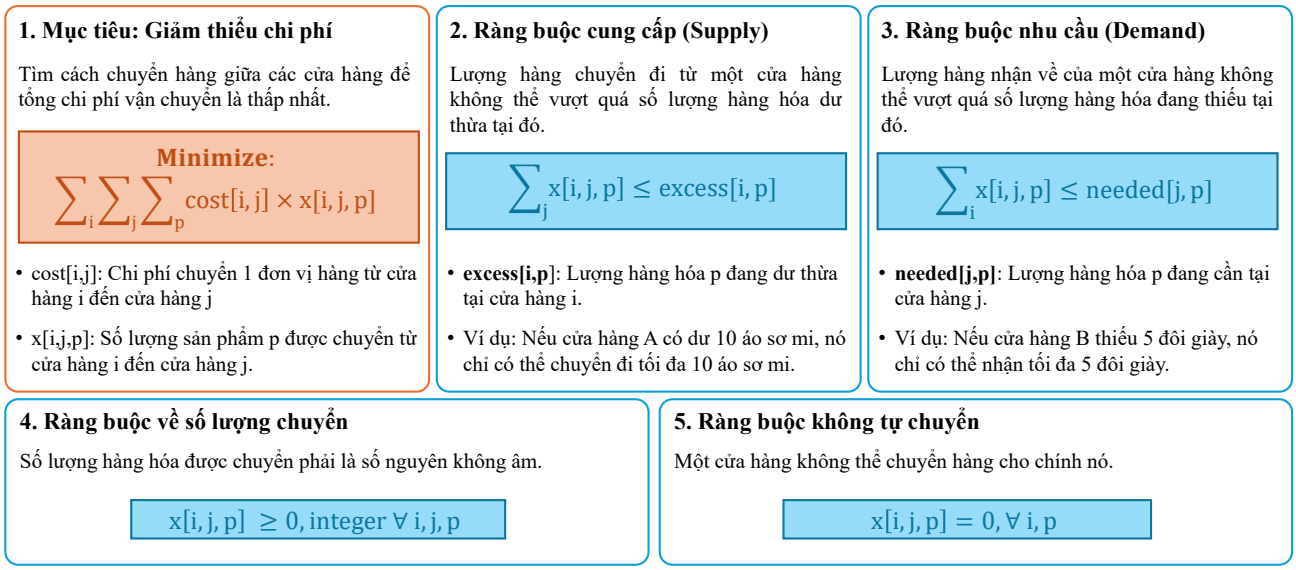
II.2. Định nghĩa bài toán

Một bài toán tối ưu về cơ bản là việc tìm ra phương án tốt nhất (ví dụ: kế hoạch luân chuyển hàng) dựa trên một bộ tiêu chí (mục tiêu) và tuân thủ các điều kiện cho trước (ràng buộc).



Hình 5: Các thành phần cốt lõi của một bài toán tối ưu:
Mục tiêu, Dữ liệu đầu vào, Ràng buộc và Kết quả đầu ra.

Để máy tính có thể hiểu và giải quyết, chúng ta cần diễn đạt bài toán này bằng ngôn ngữ toán học. Mô hình toán học giúp chúng ta lượng hóa mục tiêu và các ràng buộc một cách chính xác.



Hình 6: Mô hình toán học cho bài toán phân bổ hàng hóa nhằm tối thiểu hóa chi phí vận chuyển.

Thay vì diễn giải lại từng công thức, chúng ta sẽ tập trung vào bước quan trọng tiếp theo: làm thế nào để lập trình dự án này để có thể tìm ra lời giải. Quá trình này bao gồm ba bước chính:

1. **Cấu trúc dữ liệu đầu vào:** Đầu tiên, chúng ta cần biểu diễn các dữ liệu đầu vào dưới dạng các cấu trúc mà chương trình có thể xử lý. Trong các dự án phân tích dữ liệu với Python, cách tiếp cận hiệu quả nhất là sử dụng thư viện **Pandas**. Dữ liệu về chi phí, hàng thừa và hàng thiếu sẽ được chứa trong các đối tượng `DataFrame`.

Ví dụ cấu trúc dữ liệu trong Python với Pandas

```
# Dữ liệu về chi phí vận chuyển được tải từ file CSV
# transport_costs_df có các cột: from_store_id, to_store_id, cost
transport_costs_df.head()
#    from_store_id to_store_id   cost
# 0  CH01          CH02          15000
# 1  CH01          CH03          25000
# ...

# Dữ liệu hàng thừa và hàng thiếu được tính toán bởi InventoryAnalyzer
# và trả về dưới dạng DataFrame.
excess_df, needed_df = analyzer.identify_inventory_imbalances()

# excess_df chứa thông tin các sản phẩm đang thừa ở cửa hàng
excess_df[['store_id', 'product_id', 'excess_units']].head()
#    store_id  product_id  excess_units
# 5  CH01      AO_SO_MI    10
# 12 CH01      QUAN_JEAN   5
# ...

```

```
# needed_df chứa thông tin các sản phẩm đang thiếu ở cửa hàng
needed_df[['store_id', 'product_id', 'needed_units']].head()
#    store_id  product_id  needed_units
# 3  CH02      AO_SO_MI    8
# 9  CH03      AO_SO_MI    12
# ...
```

2. **Biểu diễn lời giải:** Thành phần chính của dự án chúng ta là cấu trúc của một "kế hoạch luân chuyển". Trong dự án này, một kế hoạch hoàn chỉnh là một danh sách các giao dịch vận chuyển. Mỗi giao dịch là một bộ thông tin xác định:

   - **Từ đâu:** Cửa hàng gửi (`from_store_id`).
   - **Đến đâu:** Cửa hàng nhận (`to_store_id`).
   - **Mặt hàng gì:** Mã sản phẩm (`product_id`).
   - **Số lượng:** Bao nhiêu đơn vị sản phẩm sẽ được chuyển (`units`).

   Đây chính là cấu trúc dữ liệu đầu ra mà các thuật toán của chúng ta phải tạo ra.

3. **Xây dựng thuật toán:** Với dữ liệu đầu vào và cấu trúc lời giải đã định nghĩa, bước cuối cùng là hiện thực hóa các thuật toán để tìm ra kế hoạch tốt nhất. Dự án này tiếp cận bài toán bằng hai phương pháp:

   - **Phương pháp dựa trên luật (Rule-Based):** Đây là một chiến lược đơn giản và trực quan. Thuật toán sẽ duyệt qua từng món hàng đang thiếu, tìm kiếm trong danh sách hàng thừa và chọn ra phương án vận chuyển có chi phí thấp nhất để đáp ứng nhu cầu đó.
   - **Phương pháp giải thuật di truyền:** Đây là một hướng tiếp cận phức tạp hơn, lấy cảm hứng từ quá trình tiến hóa tự nhiên. Thuật toán sẽ khởi tạo một "quần thể" gồm nhiều kế hoạch vận chuyển ngẫu nhiên, sau đó đánh giá, lai tạo và đột biến chúng qua nhiều thế hệ để "tiến hóa" ra lời giải tối ưu nhất dựa trên tổng chi phí.

Bằng cách chuyển đổi mô hình toán học trừu tượng thành các cấu trúc dữ liệu và thuật toán cụ thể, chúng ta đã sẵn sàng để yêu cầu máy tính tìm ra kế hoạch vận chuyển tối ưu nhất.

III. Xây dựng hệ thống

Sau khi đã có cái nhìn tổng quan về mô hình toán học và các hướng tiếp cận, chúng ta sẽ đi sâu vào chi tiết kỹ thuật của dự án. Phần này sẽ giải thích cách hệ thống được xây dựng, từ việc đọc dữ liệu thô cho đến khi tạo ra một kế hoạch luân chuyển hàng hóa hoàn chỉnh.

Sơ đồ hệ thống

```
[Dữ liệu thô]
    data/stores.csv
    data/products.csv
    data/sales_data.csv
    data/inventory_data.csv
    data/transport_cost_matrix.csv
[src/engine/analyzer.py: InventoryAnalyzer]
      1. Đọc và hợp nhất dữ liệu
      2. Tính toán tốc độ bán hàng, số ngày tồn kho
      3. Xác định hàng thừa và hàng thiếu
[Dữ liệu đã phân tích]
    excess_df (Bảng hàng thừa)
    needed_df (Bảng hàng thiếu)
[src/engine/{rule_based | genetic_algorithm}.py]
      1. Nhận excess_df, needed_df, và ma trận chi phí
      2. Áp dụng logic thuật toán
      3. Tìm kiếm kế hoạch luân chuyển
[Kết quả]
    transfer_plan_df (Kế hoạch luân chuyển tối ưu)
```

III.1. Đọc dữ liệu

Dự án sử dụng 6 file CSV trong thư mục 'data/', nhưng trong luồng phân tích chính, chúng ta chỉ làm việc trực tiếp với 5 file.

- **Các file đầu vào chính:** 'stores.csv', 'products.csv', 'sales_data.csv', 'inventory_data.csv'
- **File chi phí:** 'transport_cost_matrix.csv' chứa chi phí vận chuyển (VNĐ) giữa các cửa hàng.

Lớp `InventoryAnalyzer` sẽ đọc 4 file dữ liệu chính, trong khi file chi phí được đọc riêng để truyền vào các thuật toán. Chúng ta có cách chạy lớp InventoryAnalyzer như sau.

Đọc dữ liệu đầu vào

```python
import pandas as pd
from src.engine.analyzer import InventoryAnalyzer

# Khởi tạo analyzer
analyzer = InventoryAnalyzer()

# Tải 4 file dữ liệu kinh doanh cốt lõi
analyzer.load_data(
    sales_path="data/sales_data.csv",
    inventory_path="data/inventory_data.csv",
    stores_path="data/stores.csv",
    products_path="data/products.csv"
```

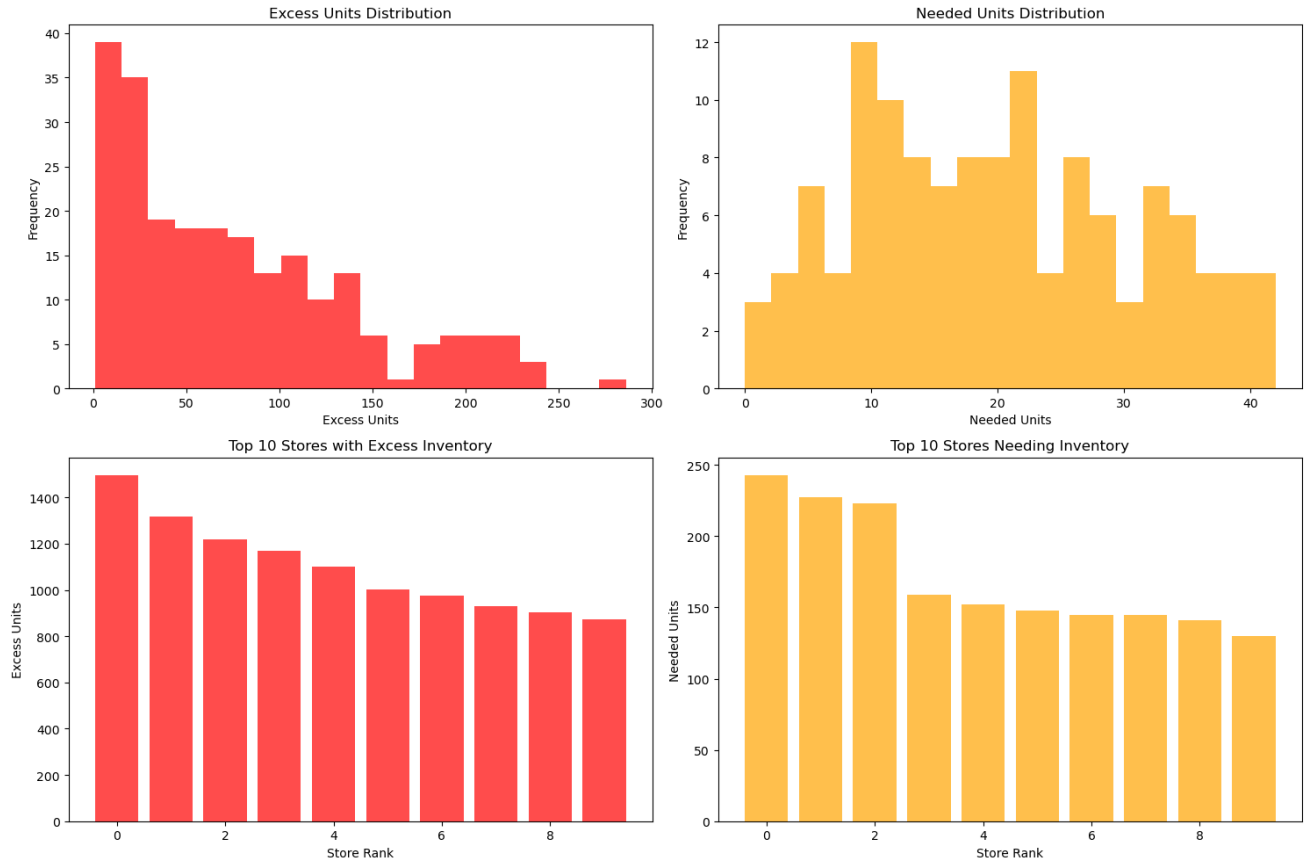
```
)

# Dữ liệu chi phí vận chuyển cũng được đọc riêng
transport_costs_df = pd.read_csv("data/transport_cost_matrix.csv")
```

Sau khi tải, phương thức `identify_inventory_imbalances` sẽ tạo ra hai "bảng" dữ liệu, là đầu vào cốt lõi cho cả hai thuật toán tối ưu.

- **Bảng hàng thừa ('excess_df'):** Liệt kê những sản phẩm tại các cửa hàng có lượng tồn kho vượt quá MAX_INVENTORY_DAYS. Bảng này sẽ có thêm cột excess_units cho biết số lượng đơn vị đang thừa.
- **Bảng hàng thiếu ('needed_df'):** Liệt kê những sản phẩm tại các cửa hàng có lượng tồn kho thấp hơn MIN_INVENTORY_DAYS. Bảng này có cột needed_units cho biết số lượng cần bổ sung.

Với hai bảng dữ liệu này cùng với ma trận chi phí vận chuyển, chúng ta đã có đầy đủ thông tin cần thiết để bắt đầu tìm kiếm một kế hoạch luân chuyển hàng hóa hiệu quả.



Hình 7: Phân phối của dữ liệu. Ta thấy rằng số lượng hàng dư ít hơn số lượng hàng cần rất nhiều. Đây là một bộ dữ liệu bị mất cân bằng.

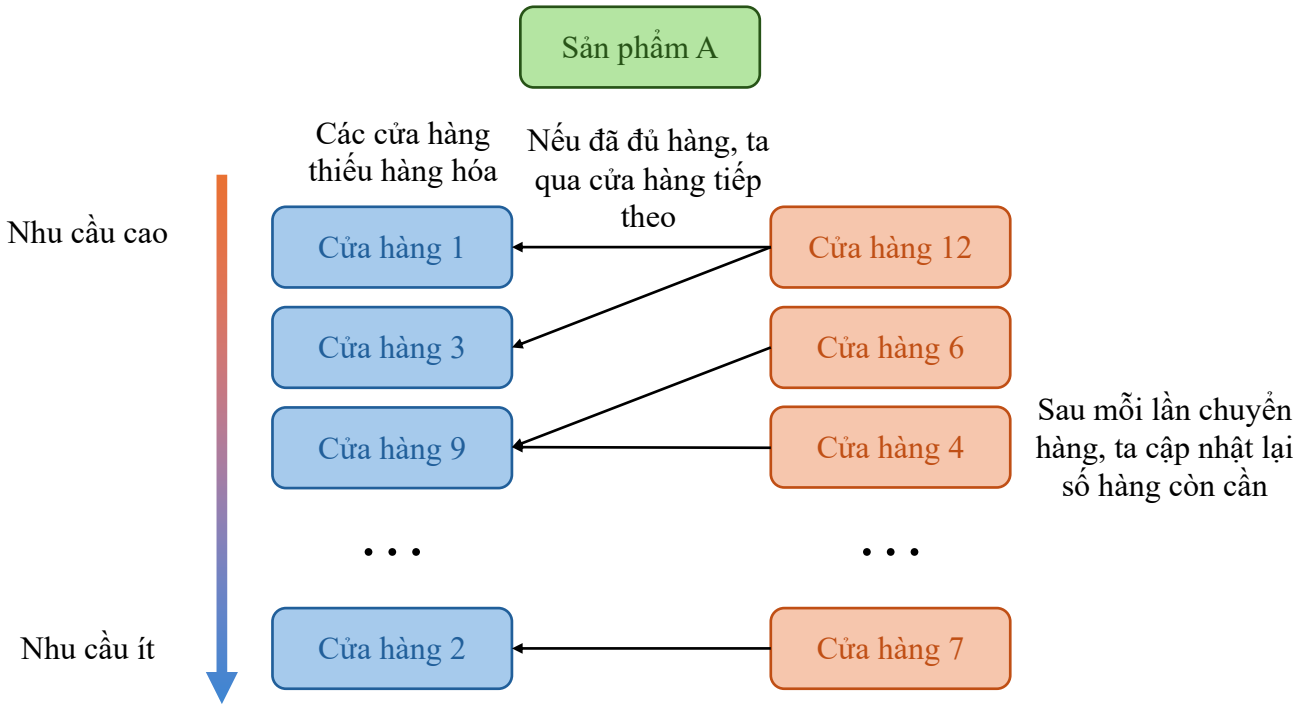
III.2. Phương pháp dựa trên các luật

Đây là phương pháp tiếp cận đầu tiên và đơn giản nhất để giải quyết bài toán. Ý tưởng cốt lõi của nó là một chiến lược "tham lam" (greedy): với mỗi món hàng đang thiếu, chúng ta sẽ ngay lập tức tìm cách đáp ứng nó bằng nguồn cung có chi phí vận chuyển rẻ nhất. Phương pháp này không đảm bảo sẽ tìm ra lời giải tối ưu nhất trên toàn cục, nhưng nó rất nhanh, dễ hiểu và thường cho ra một kết quả "đủ tốt".

Logic hoạt động Thuật toán được thực hiện trong lớp `RuleBasedOptimizer`. Luồng của thuật toán có thể được mô tả như sau:

1. **Sắp xếp nhu cầu:** Đầu tiên, thuật toán sắp xếp bảng `needed_df` để ưu tiên những nhu cầu lớn nhất. Điều này giúp giải quyết các vấn đề thiếu hụt nghiêm trọng trước.
2. **Duyệt qua từng nhu cầu:** Với mỗi dòng trong bảng `needed_df` (tức là mỗi sản phẩm `p` đang thiếu tại cửa hàng `j`), thuật toán sẽ:
   (a) Tìm tất cả các cửa hàng `i` có thừa sản phẩm `p` từ bảng `excess_df`.
   (b) Với mỗi cửa hàng `i` tìm được, lấy ra chi phí vận chuyển từ `i` đến `j` trong ma trận chi phí.
   (c) Sắp xếp các cửa hàng cung cấp tiềm năng này theo chi phí vận chuyển tăng dần.
3. **Thực hiện vận chuyển:** Thuật toán bắt đầu duyệt qua danh sách các nguồn cung đã sắp xếp (từ rẻ nhất đến đắt nhất). Nó sẽ lấy hàng từ nguồn rẻ nhất cho đến khi nhu cầu được đáp ứng, hoặc nguồn cung đó hết hàng. Nếu nhu cầu vẫn chưa đủ, nó sẽ tiếp tục lấy từ nguồn rẻ thứ hai, và cứ thế tiếp tục.
4. **Cập nhật và lặp lại:** Sau mỗi lần vận chuyển, số lượng hàng thừa và thiếu ở các kho liên quan sẽ được cập nhật lại. Thuật toán tiếp tục cho đến khi tất cả các nhu cầu đã được xem xét.

Logic trên có thể được biểu diễn bằng đoạn mã giả sau, giúp làm rõ cấu trúc của vòng lặp chính trong phương thức `optimize`. Ở đây, ta có thể thấy thuật toán có độ phức tạp là $O(N \times M \times log(M))$ với N là số lượng như cầu, và M là số lượng hàng thừa.



Hình 8: Hình minh họa thuật toán rule-based dựa trên một sản phẩm. Tiêu chí chúng ta chọn cửa hàng cung cấp hàng hóa cần là chi phí vận chuyển thấp nhất.

Thuật toán Rule-Based

```python
def rule_based_optimizer(excess_df, needed_df, transport_cost_matrix):

    transfer_plan = []

    # Tạo bản sao để cập nhật số lượng mà không ảnh hưởng đến DataFrame gốc
    excess_inventory = excess_df.set_index(['store_id', 'product_id'])

    # Sắp xếp các nhu cầu từ lớn nhất đến nhỏ nhất
    needed_sorted = needed_df.sort_values(by='needed_units', ascending=False)

    # Vòng lặp chính: duyệt qua từng nhu cầu
    for _, need_row in needed_sorted.iterrows():
        product_id = need_row['product_id']
        to_store = need_row['to_store_id']
        needed_units = need_row['needed_units']

        # Tìm các nguồn cung có cùng sản phẩm
        supplies = excess_inventory.query(f"product_id == '{product_id}'")

        if supplies.empty:
            continue

        # Tính chi phí vận chuyển từ mỗi nguồn cung đến nơi cần
```

```
        supplies['cost'] = supplies.index.map(
            lambda from_store: transport_cost_matrix.loc[from_store, to_store]
        )

        # Sắp xếp các nguồn cung theo chi phí rẻ nhất
        supplies = supplies.sort_values(by='cost', ascending=True)

        # Bắt đầu đáp ứng nhu cầu từ nguồn cung rẻ nhất
        for from_store, supply_row in supplies.iterrows():
            available_units = supply_row['excess_units']

            # Xác định số lượng có thể chuyển
            transfer_units = min(needed_units, available_units)

            if transfer_units > 0:
                # Ghi nhận một giao dịch
                transfer_plan.append({
                    'from_store_id': from_store,
                    'to_store_id': to_store,
                    'product_id': product_id,
                    'units': transfer_units,
                    'transport_cost': supply_row['cost'] * transfer_units
                })

                # Cập nhật lại số lượng
                excess_inventory.loc[from_store, 'excess_units'] -= transfer_units
                needed_units -= transfer_units

                # Nếu đã đủ hàng, chuyển sang xử lý nhu cầu tiếp theo
                if needed_units == 0:
                    break

    return pd.DataFrame(transfer_plan)
```

Sau khi đã định nghĩa thuật toán xong, việc sử dụng thuật toán này rất đơn giản. Sau khi đã có `excess_df` và `needed_df` từ `InventoryAnalyzer`, chúng ta chỉ cần khởi tạo `RuleBasedOptimizer` và gọi phương thức `optimize`.

Thực thi thuật toán Rule-Based

```
# Khởi tạo Rule-Based Optimizer với ma trận chi phí
rule_optimizer = RuleBasedOptimizer(
    transport_cost_matrix=transport_cost_matrix
)

# Chạy thuật toán
rule_transfer_plan = rule_optimizer.optimize(excess_df, needed_df)

# In ra kết quả
print(f"Tổng chi phí vận chuyển: {rule_transfer_plan['transport_cost'].sum():,.0f} VND"
                                                                                )
display(rule_transfer_plan.head())
```

Kết quả trả về là một DataFrame chứa kế hoạch chi tiết: sản phẩm nào cần được chuyển, từ đâu, đến đâu, với số lượng và chi phí là bao nhiêu. Đây là một giải pháp nhanh chóng và dễ dàng để cải thiện tình hình tồn kho.

idx	from_store_id	to_store_id	product_id	units	distance (km)	transport_cost (VND)
0	5	7	3	42	20.53	2.07×10^6
1	19	14	23	42	6.79	6.84×10^5
2	5	2	4	42	7.41	7.47×10^5
3	18	15	24	40	6.20	5.95×10^5
4	19	17	19	39	2.69	2.52×10^5
5	19	20	19	19	12.91	5.89×10^5
6	14	20	19	20	14.61	7.01×10^5
7	8	10	25	22	3.34	1.76×10^5
8	20	10	25	16	596.07	1.14×10^7
9	8	15	28	38	611.75	2.79×10^7

Bảng 1: Ví dụ về kết quả đầu ra của kế hoạch vận chuyển. Với mỗi hàng là kết hoạch vận chuyển của một sản phẩm, và một sản phẩn có thể được vận chuyển nhiều lần.

III.3. Phương pháp thuật toán di truyền

Nếu phương pháp dựa trên luật là một chiến lược đơn giản và nhanh chóng, thì thuật toán di truyền (GA) là một cách tiếp cận phức tạp hơn, được thiết kế để khám phá không gian lời giải một cách rộng hơn nhằm tìm ra kết quả tiệm cận tối ưu toàn cục. Thuật toán này mô phỏng lại quá trình tiến hóa tự nhiên: tạo ra một "quần thể" các lời giải, cho chúng "giao phối" và "đột biến" qua nhiều thế hệ để tìm ra lời giải tốt nhất (có chi phí thấp nhất).

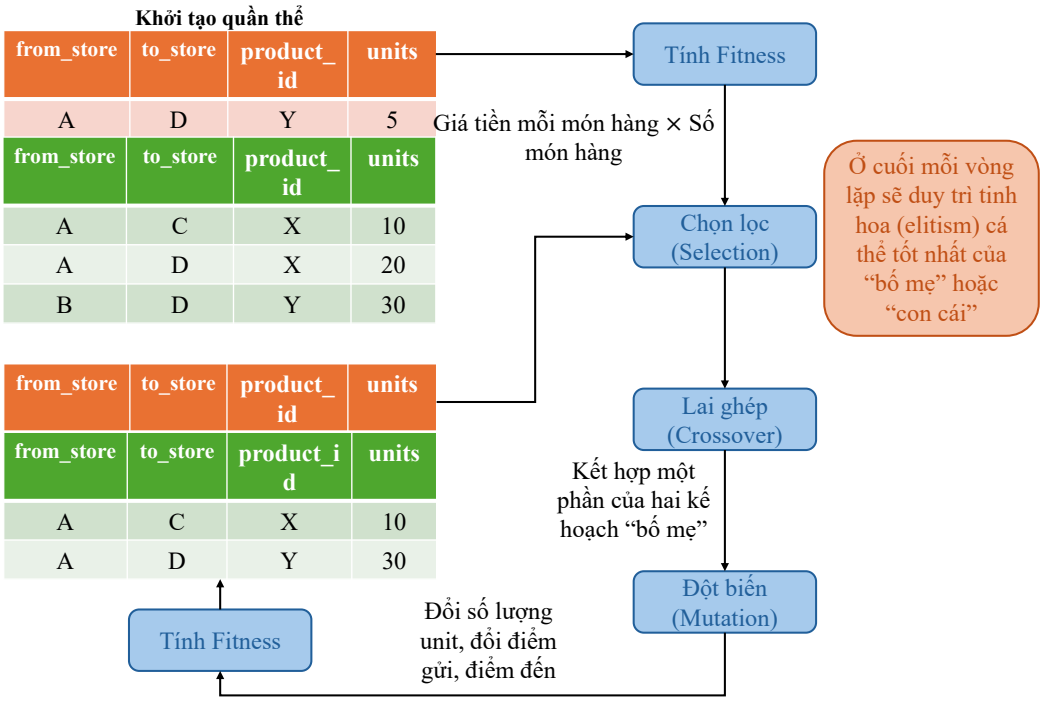
Thuật toán được xây dựng trong lớp `GeneticAlgorithmOptimizer`. Để hiểu nó, chúng ta cần nắm các khái niệm sau:

- **Cá thể (Individual):** Mỗi cá thể là một lời giải hoàn chỉnh, tức là một kế hoạch luân chuyển hàng hóa.
- **Quần thể (Population):** Là một tập hợp gồm nhiều cá thể. Ví dụ, một quần thể có 50 cá thể tức là 50 kế hoạch luân chuyển khác nhau.
- **Hàm thích nghi (Fitness Function):** Đây là hàm đánh giá chất lượng của một cá thể. Trong bài toán này, "thích nghi" đồng nghĩa với "chi phí thấp". Hàm này tính tổng chi phí vận chuyển của một kế hoạch. Cá thể nào có fitness thấp hơn thì được coi là tốt hơn.
- **Chọn lọc (Selection):** Quá trình chọn ra các cá thể "tốt" (chi phí thấp) từ quần thể để làm "cha mẹ" cho thế hệ tiếp theo. Dự án sử dụng phương pháp *Tournament Selection*.
- **Lai ghép (Crossover):** Quá trình hai cha mẹ "giao phối" để tạo ra con. Về bản chất, nó trộn lẫn các phần của hai kế hoạch vận chuyển để tạo ra một kế hoạch mới, hy vọng sẽ tốt hơn.

- **Đột biến (Mutation):** Thay đổi nhỏ, ngẫu nhiên trên một cá thể con để tạo ra sự đa dạng trong quần thể, tránh việc bị "mắc kẹt" ở một lời giải cục bộ.

Để vận hành thuật toán, chúng ta cần tinh chỉnh các tham số cốt lõi như sau. *Kích thước quần thể (population size)* quyết định số lượng lời giải được duy trì trong mỗi vòng lặp, trong khi *số thế hệ (generations)* xác định độ dài của quá trình tiến hóa. Bên cạnh đó, *tỷ lệ lai ghép (crossover rate)* và *tỷ lệ đột biến (mutation rate)* là hai xác suất quan trọng, điều khiển tần suất các cá thể mới được tạo ra thông qua việc kết hợp từ cha mẹ hoặc thay đổi ngẫu nhiên, từ đó cân bằng giữa việc khai thác các lời giải tốt hiện có và khám phá những vùng lời giải tiềm năng mới. Các tham số được nói trên có thể được tin chỉnh ở file 'src/config.py'.

Hàm thích nghi (`_calculate_fitness`) Đây là hàm lõi để đánh giá mỗi kế hoạch vận chuyển. Logic của nó dựa trên hàm mục tiêu mà ta đã định nghĩa ở trên: duyệt qua từng hàng trong kế hoạch, tra cứu chi phí trên mỗi đơn vị từ ma trận chi phí, nhân với số lượng, rồi cộng dồn lại.



Hình 9: Quá trình thuật toán Genetic sẽ thực hiện trong hệ thống.

Giả mã hàm `_calculate_fitness`

```
def _calculate_fitness(transfer_plan, transport_cost_matrix):
    total_cost = 0

    for transfer in transfer_plan:
        from_store = transfer['from_store']
        to_store = transfer['to_store']
        units = transfer['units']
```

```
        # Tra cứu chi phí trên mỗi đơn vị
        cost_per_unit = transport_cost_matrix.loc[from_store, to_store]

        # Cộng dồn vào tổng chi phí
        total_cost += cost_per_unit * units

    return total_cost
```

Lai ghép (`_crossover`) Hàm này nhận hai kế hoạch (cha và mẹ) và tạo ra hai kế hoạch mới (con). Dự án sử dụng phương pháp lai ghép một điểm (single-point crossover) đơn giản.

Giả mã hàm `_crossover`

```
def _crossover(parent1_plan, parent2_plan):
    # Chọn một điểm ngẫu nhiên để cắt
    crossover_point = random.randint(1, min(len(parent1_plan), len(parent2_plan)) - 1)

    # Tạo ra hai cá thể con
    # Con 1 = đầu của cha 1 + đuôi của cha 2
    child1_plan = parent1_plan[:crossover_point] + parent2_plan[crossover_point:]

    # Con 2 = đầu của cha 2 + đuôi của cha 1
    child2_plan = parent2_plan[:crossover_point] + parent1_plan[crossover_point:]

    # "Sửa chữa" các kế hoạch con để đảm bảo chúng hợp lệ
    child1_plan = _repair_solution(child1_plan)
    child2_plan = _repair_solution(child2_plan)

    return child1_plan, child2_plan
```

Đột biến (`_mutate`) Để duy trì sự đa dạng, một cá thể con có thể bị đột biến ngẫu nhiên. Hàm `_mutate` sẽ chọn một trong bốn loại đột biến sau:

Giả mã các loại đột biến trong hàm `_mutate`

```
def _mutate(individual_plan):
    mutation_type = random.choice(['change_quantity', 'remove_transfer', 'add_transfer'
                                   , 'change_route'])

    if mutation_type == 'change_quantity':
        # 1. Thay đổi số lượng của một giao dịch ngẫu nhiên
        random_transfer = random.choice(individual_plan)
        max_possible_units = get_max_transfer(random_transfer['from_store'], ...)
        random_transfer['units'] = random.randint(1, max_possible_units)

    elif mutation_type == 'remove_transfer':
        # 2. Xóa một giao dịch ngẫu nhiên
        if len(individual_plan) > 1:
```

```
            individual_plan.pop(random.randint(0, len(individual_plan) - 1))

    elif mutation_type == 'add_transfer':
        # 3. Thêm một giao dịch hoàn toàn mới và ngẫu nhiên
        new_transfer = _create_random_transfer()
        if new_transfer:
            individual_plan.append(new_transfer)

    elif mutation_type == 'change_route':
        # 4. Thay đổi nguồn hoặc đích của một giao dịch
        random_transfer = random.choice(individual_plan)
        if random.random() < 0.5:
            # Thay đổi cửa hàng gửi (from_store)
            new_from_store = find_another_store_with_excess(random_transfer['product_id
                                                            '])
            random_transfer['from_store'] = new_from_store
        else:
            # Thay đổi cửa hàng nhận (to_store)
            new_to_store = find_another_store_with_need(random_transfer['product_id'])
            random_transfer['to_store'] = new_to_store

    # Cuối cùng, sửa chữa lại kế hoạch để đảm bảo tính hợp lệ
    individual_plan = _repair_solution(individual_plan)
    return individual_plan
```

Tương tự như thuật toán trước, việc gọi GA cũng rất đơn giản. Chúng ta khởi tạo lớp GAOptimizer và gọi phương thức `optimize`, truyền vào các tham số cấu hình như kích thước quần thể, số thế hệ, v.v. Ta có thể thấy rằng với thiết lập của thuật toán này, thời gian cần để đưa ra đáp án sẽ nhiều hơn đáng kể so với thuật toán Rule-based được nói ở trên.

Thực thi thuật toán Genetic Algorithm

```
# Khởi tạo GA Optimizer
ga_optimizer = GeneticAlgorithmOptimizer(
    transport_cost_matrix=transport_cost_matrix
)

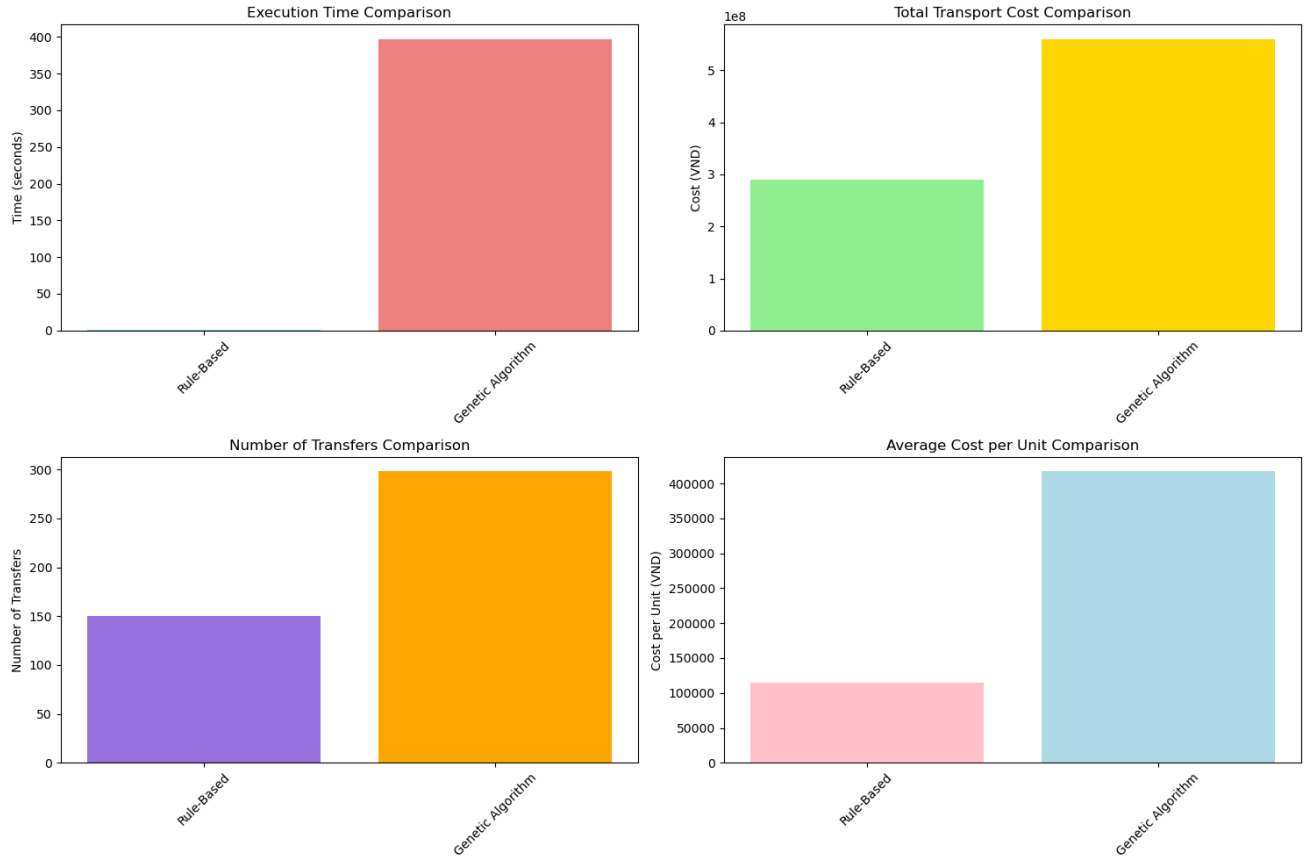
# Chạy thuật toán với các tham số từ file config
ga_transfer_plan = ga_optimizer.optimize(
    excess_inventory=excess_df,
    needed_inventory=needed_df,
    population_size=GA_POPULATION_SIZE,
    num_generations=GA_GENERATIONS,
    verbose=True  # Hiển thị tiến trình qua các thế hệ
)

# In ra kết quả
print(f"Tổng chi phí vận chuyển (GA): {ga_transfer_plan['transport_cost'].sum():,.0f}
                                       VND")
display(ga_transfer_plan.head())
```

Quá trình này sẽ mất nhiều thời gian hơn so với phương pháp dựa trên luật, nhưng nó có khả năng tìm ra một kế hoạch luân chuyển với tổng chi phí thấp hơn đáng kể.

III.4. So sánh hiệu năng và lựa chọn thuật toán

Sau khi đã hiện thực hóa cả hai phương pháp, chúng ta cần so sánh và lựa chọn ra chiến lược tối ưu nhất cho bài toán hiện tại. Dựa trên kết quả thực thi được ghi nhận trong notebook (xem Hình 10), chúng ta có những kết luận như sau.



Hình 10: Biểu đồ so sánh hiệu năng giữa thuật toán Rule-Based và thuật toán di truyền trên bốn tiêu chí: thời gian thực thi, tổng chi phí, số lượng chuyến hàng, và chi phí trung bình trên mỗi đơn vị.

Về **thời gian thực thi**, sự khác biệt của hai thuật toán là rất lớn: thuật toán Rule-Based hoàn thành gần như tức thì chỉ trong vài giây, trong khi thuật toán di truyền đòi hỏi năng lực tính toán lớn và mất vài trăm giây để chạy qua tất cả các thế hệ (thay đổi mỗi lần chạy). Tuy nhiên, kết quả bất ngờ và quan trọng nhất nằm ở **tổng chi phí vận chuyển**. Thuật toán Rule-Based, với logic "tham lam" đơn giản, lại tìm ra một kế hoạch có tổng chi phí thấp hơn đáng kể so với thuật toán di truyền. Điều này cho thấy chiến lược "ưu tiên tuyến đường rẻ nhất" tỏ ra rất hiệu quả với bộ dữ liệu hiện tại. Phân tích sâu hơn về **số lượng chuyến hàng**, thuật toán di truyền có xu hướng tạo ra một kế hoạch "phân mảnh" hơn với số lượng giao dịch gần

như gấp đôi, trong khi Rule-Based lại gộp các giao dịch hiệu quả hơn. Cuối cùng, **chi phí trung bình trên mỗi đơn vị** một lần nữa khẳng định điều này, khi chi phí của GA cao hơn nhiều so với Rule-Based, chứng tỏ các quyết định của thuật toán đơn giản lại hiệu quả hơn về mặt chi phí.

Vì sao Rule-Based lại hiệu quả hơn?

Lý giải cho kết quả nói trên nằm ở cấu trúc rất rõ ràng của bài toán. Luồng xử lý được định nghĩa rất cụ thể: mỗi sản phẩm chỉ được phép di chuyển từ một cửa hàng đang thừa sang một cửa hàng đang thiếu, và phải tuân thủ ràng buộc chặt chẽ là không được vận chuyển vượt quá số lượng tồn kho. Mục tiêu duy nhất là tối thiểu hóa chi phí.
Nhờ cấu trúc chặt chẽ được nói trên, phương pháp Rule-Based có thể tận dụng logic đặc thù của bài toán một cách triệt để, giúp nó nhanh chóng tìm ra lời giải tốt, dễ kiểm soát và mang lại hiệu quả cao.

IV. Hướng cải tiến

Hệ thống hiện tại đã cung cấp một giải pháp hoàn chỉnh cho bài toán luân chuyển tồn kho. Tuy nhiên, trong một kịch bản vận hành thực tế, luôn có những cơ hội để cải tiến và tối ưu hơn nữa. Dưới đây là hai hướng đi tiềm năng có thể giúp nâng cao đáng kể hiệu quả của hệ thống.

IV.1. Tích hợp dữ liệu thực và mô hình dự báo

Một trong những hạn chế lớn nhất của hệ thống hiện tại là nó mang tính "phản ứng" (reactive). Nó xác định hàng thừa và thiếu dựa trên tốc độ bán hàng *trong quá khứ* được ghi ở trong file "sales_data.csv". Một cải tiến mà chúng ta có thể thực hiện ở đây chuyển hệ thống sang phương pháp "chủ động" (proactive) bằng cách tích hợp các mô hình dự báo (forecasting models) để dự đoán nhu cầu tương lai.

Ví dụ với dữ liệu thực tế Để đưa hệ thống đến gần hơn với thực tế, chúng ta có thể sử dụng các bộ dữ liệu bán lẻ công khai. Mặc dù dữ liệu nội bộ của doanh nghiệp là không có sẵn, chúng ta có thể dùng một bộ dữ liệu tương đương là **Brazilian E-Commerce Public Dataset by Olist**.

Áp dụng bộ dữ liệu Olist vào dự án

Nguồn dữ liệu: Brazilian E-Commerce Public Dataset by Olist trên Kaggle.

Bộ dữ liệu này chứa thông tin của gần 100,000 đơn hàng và có thể được ánh xạ vào dự án của chúng ta như sau:

- **Dữ liệu bán hàng ('sales_data.csv'):** Có thể được tổng hợp từ file `olist_order_items_dataset.csv`. Mỗi `seller_id` sẽ được coi là một "cửa hàng".
- **Dữ liệu sản phẩm ('products.csv'):** Lấy trực tiếp từ file `olist_products_dataset.csv`.
- **Dữ liệu cửa hàng ('stores.csv'):** Lấy từ `olist_sellers_dataset.csv`, trong đó mỗi người bán (`seller_id`) là một cửa hàng.
- **Ma trận khoảng cách và chi phí:** File `olist_geolocation_dataset.csv` cung cấp tọa độ địa lý thực của các mã zip cho mỗi người bán ở trên, cho phép chúng ta tính toán ma trận khoảng cách và chi phí vận chuyển cho bài toán hiện tại.
- **Dữ liệu tồn kho ('inventory_data.csv'):** Đây là phần duy nhất bị thiếu của bộ dữ liệu E-Commerce nói trên. Chúng ta cần tự tạo ra nó (synthesize) bằng cách tính tốc độ bán hàng trung bình của mỗi sản phẩm tại mỗi cửa hàng, sau đó gán một mức tồn kho ngẫu nhiên nhưng hợp lý (ví dụ: tồn kho bằng 20-100 ngày bán hàng). Các bạn có thể tham khảo folder 'src/data_generator' trong bài toán để có thể biết cách tạo dữ liệu ở đây.

Việc sử dụng bộ dữ liệu này giúp kiểm thử các thuật toán trên một quy mô lớn hơn và với các đặc tính phân phối địa lý thực tế.

Sau khi có dữ liệu bán hàng theo chuỗi thời gian từ Olist, chúng ta có thể huấn luyện các mô hình dự báo như **ARIMA**, **Prophet**, hoặc **LSTM**. Thay vì tính `avg_daily_sales` dựa trên quá khứ, chúng ta sẽ dùng `forecasted_daily_sales` cho 30 ngày tới. Từ đó, hệ thống có thể xác định các sản phẩm *sẽ sớm* bị thiếu hoặc thừa, phù hợp hơn trong các bài toán thực tế của chúng ta.

IV.2. Cải tiến và tối ưu hóa giải thuật di truyền

Bên cạnh việc tích hợp dữ liệu dự báo, bản thân thuật toán di truyền cũng có rất nhiều vị trí cần để cải thiện, giúp nó tìm ra lời giải tốt hơn cho bài toán đơn mục tiêu hiện tại. Hướng cải tiến đầu tiên và cơ bản nhất chính là **tối ưu hóa tham số**. Các tham số như kích thước quần thể, tỷ lệ lai ghép, và tỷ lệ đột biến có ảnh hưởng trực tiếp đến sự cân bằng giữa tốc độ hội tụ và khả năng khám phá không gian lời giải. Việc đánh giá và so sánh kết quả khi thay đổi từng tham số sẽ giúp chúng ta hiểu rõ mức độ ảnh hưởng của chúng đến hiệu quả thuật toán.

Các hướng nâng cấp cấu trúc cho thuật toán i truyền

Khi bài toán trở nên phức tạp hơn, chúng ta có thể xem xét các nâng cấp sâu hơn về mặt cấu trúc như sau:

- **Cải tiến khởi tạo quần thể:** Thay vì khởi tạo hoàn toàn ngẫu nhiên, ta có thể tạo ra một quần thể ban đầu "thông minh" hơn bằng cách kết hợp một tỷ lệ nhỏ (ví dụ: 20%) các cá thể tốt được tạo ra từ phương pháp Rule-Based với phần còn lại (80%) được tạo ngẫu nhiên.
- **Thiết kế toán tử lai ghép đặt biệt cho bài toán này (Problem-aware Crossover):** Phép lai ghép mặc định có thể phá vỡ các tuyến vận chuyển tốt đang có. Một phép lai ghép được thiết kế riêng cho bài toán (ví dụ: route-preserving crossover) sẽ giữ lại các chuỗi quyết định hợp lý từ cặp "cha mẹ".
- **Sử dụng toán tử thích ứng (Adaptive Operators):** Thay vì giữ cố định các tỷ lệ như tỉ lệ đột biến, tỉ lệ lai, . . . , thuật toán có thể tự động điều chỉnh chúng trong quá trình chạy. Ví dụ, giảm tỷ lệ đột biến khi thuật toán gần hội tụ để tinh chỉnh lời giải một cách chính xác hơn.
- **Sử dụng phương pháp lai ghép (Hybrid Approach):** Đây là một kỹ thuật kết hợp GA với một thuật toán tìm kiếm cục bộ (local search). Sau mỗi thế hệ, những cá thể tốt nhất sẽ được một thuật toán cục bộ tối ưu hóa thêm. Cách tiếp cận này còn được gọi là **Memetic Algorithm** và đã được áp dụng hiệu quả trong dự án thực tế.

Những cải tiến này đặc biệt phù hợp khi bài toán có **quy mô lớn** (ví dụ: trên 500 cửa hàng, 1,000 sản phẩm) hoặc có các **ràng buộc động** thay đổi theo thời gian (chi phí, giới hạn kho, điều kiện vận hành), khiến cho các lời giải đơn giản không còn hiệu quả.

IV.3. Áp dụng thuật toán đa mục tiêu (NSGA-II)

Thuật toán di truyền hiện tại được thiết kế để tối ưu một mục tiêu duy nhất: **tổng chi phí vận chuyển**. Tuy nhiên, trên thực tế, một doanh nghiệp có thể muốn cân bằng nhiều mục tiêu khác nhau và chúng thường mâu thuẫn với nhau. Ví dụ:

- **Giảm chi phí vận chuyển** (mục tiêu hiện tại).
- **Giảm số lượng chuyến hàng** (giảm chi phí nhân công, vận hành).
- **Tối đa hóa mức độ đáp ứng nhu cầu** (fill rate).
- **Cân bằng tồn kho sau luân chuyển** (giảm rủi ro trên toàn hệ thống).

Để giải quyết bài toán này, chúng ta có thể sử dụng một biến thể của thuật toán di truyền có tên là **NSGA-II (Non-dominated Sorting Genetic Algorithm II)**. Thay vì tìm ra một lời giải "tốt nhất", NSGA-II sẽ tìm ra một tập hợp các lời giải tối ưu được gọi là **mặt trận Pareto (Pareto Front)**. Mỗi lời giải trên mặt trận này đại diện cho một sự đánh đổi khác nhau giữa các mục tiêu (ví dụ: một lời giải chi phí thấp nhưng đáp ứng nhu cầu thấp, một lời giải khác chi

phí cao nhưng đáp ứng nhu cầu cao). Điều này cho phép người quản lý lựa chọn kịch bản phù hợp nhất với chiến lược kinh doanh tại từng thời điểm.

V. Câu hỏi trắc nghiệm

1. Trong file `src/config.py`, biến `MIN_INVENTORY_DAYS` được sử dụng để làm gì?
   (*a*) Xác định tuổi tối thiểu của một sản phẩm để được coi là hàng tồn kho.
   (*b*) Tính toán mức tồn kho an toàn tối thiểu mà một cửa hàng nên duy trì cho mỗi sản phẩm.
   (*c*) Quy định thời gian vận chuyển tối thiểu giữa hai cửa hàng bất kỳ.
   (*d*) Đặt ra số ngày tối thiểu mà một sản phẩm phải có trong kho trước khi được bán.

2. Lớp `InventoryAnalyzer` trong file `src/engine/analyzer.py` xác định một sản phẩm tại một cửa hàng là "hàng thừa" (excess) dựa trên tiêu chí chính nào?
   (*a*) Khi số lượng tồn kho của nó lớn hơn tổng nhu cầu của toàn bộ hệ thống.
   (*b*) Khi chỉ số `days_of_supply` của nó cao hơn giá trị `MAX_INVENTORY_DAYS`.
   (*c*) Khi chỉ số `days_of_supply` của nó thấp hơn giá trị `MIN_INVENTORY_DAYS`.
   (*d*) Khi sản phẩm đó không bán được trong suốt `MAX_INVENTORY_DAYS` ngày.

3. Logic cốt lõi của thuật toán `RuleBasedOptimizer` là gì?
   (*a*) Nó tạo ra các cặp điều chuyển ngẫu nhiên giữa nơi thừa và nơi thiếu.
   (*b*) Nó ưu tiên thực hiện các lệnh điều chuyển có số lượng lớn nhất trước.
   (*c*) Đây là một thuật toán tham lam (greedy), với mỗi mặt hàng đang thiếu, nó sẽ tìm nguồn hàng thừa có chi phí vận chuyển rẻ nhất để đáp ứng.
   (*d*) Nó xây dựng một đồ thị phức tạp và tìm đường đi ngắn nhất cho tất cả các lệnh điều chuyển.

4. Trong thuật toán di truyền (`GeneticAlgorithmOptimizer`), hàm `_calculate_fitness` đánh giá một "cá thể" dựa trên tiêu chí nào là chính?
   (*a*) Số lượng đơn vị hàng hóa được vận chuyển nhiều nhất.
   (*b*) Thời gian thực thi nhanh nhất của giải pháp.
   (*c*) Tổng chi phí vận chuyển của toàn bộ kế hoạch điều chuyển (càng thấp càng tốt).
   (*d*) Số lượng cửa hàng tham gia vào quá trình điều chuyển.

5. Tại sao thuật toán di truyền (GA) thường tìm ra giải pháp có tổng chi phí thấp hơn so với thuật toán dựa trên luật (Rule-based)?
   (*a*) Vì GA luôn chạy trong thời gian lâu hơn nên có nhiều thời gian để tính toán hơn.
   (*b*) Vì GA có khả năng thoát khỏi các "tối ưu cục bộ" bằng cách khám phá một không gian giải pháp rộng lớn hơn thông qua lai ghép và đột biến.
   (*c*) Vì GA luôn thực hiện ít lệnh điều chuyển hơn, do đó giảm chi phí.
   (*d*) Vì thuật toán dựa trên luật không tính đến chi phí vận chuyển.

6. Trong file `notebooks/01_full_process_step_by_step.ipynb`, mục đích chính của việc chạy hai thuật toán và so sánh kết quả là gì?

(*a*) Để chứng minh rằng cả hai thuật toán đều cho ra cùng một kết quả chính xác.

(*b*) Để kiểm tra xem Python đã được cài đặt đúng hay chưa.

(*c*) Để đánh giá sự đánh đổi (trade-off) giữa tốc độ thực thi (Rule-based nhanh hơn) và chất lượng giải pháp (GA thường có chi phí thấp hơn).

(*d*) Để xác định xem nên sử dụng Pandas hay NumPy cho việc phân tích.

7. Hàm `_repair_solution` trong `GeneticAlgorithmOptimizer` có vai trò gì?

   (*a*) Sửa các lỗi cú pháp trong mã nguồn của thuật toán.

   (*b*) Đảm bảo rằng một giải pháp (sau khi lai ghép hoặc đột biến) không vi phạm các ràng buộc cơ bản, ví dụ như chuyển nhiều hơn số lượng đang có.

   (*c*) Tối ưu hóa lại chi phí vận chuyển sau khi đã có giải pháp.

   (*d*) Làm tròn số lượng hàng hóa vận chuyển thành số nguyên gần nhất.

8. Dữ liệu trong file `transport_cost_matrix.csv` biểu diễn điều gì?

   (*a*) Chi phí để vận chuyển một đơn vị sản phẩm giữa hai cửa hàng bất kỳ.

   (*b*) Khoảng cách (km) giữa các cửa hàng.

   (*c*) Thời gian vận chuyển trung bình (giờ) giữa các cửa hàng.

   (*d*) Chi phí lưu kho cho mỗi sản phẩm tại mỗi cửa hàng.

9. Trong thuật toán di truyền, toán tử "lai ghép" (Crossover) thực hiện nhiệm vụ gì?

   (*a*) Thay đổi ngẫu nhiên một vài gen trong một cá thể để tạo sự đa dạng.

   (*b*) Đánh giá chất lượng của từng cá thể trong quần thể.

   (*c*) Kết hợp thông tin từ hai cá thể "cha mẹ" để tạo ra các cá thể "con" mới, kế thừa các đặc tính tốt.

   (*d*) Lựa chọn những cá thể tốt nhất từ quần thể hiện tại để giữ lại cho thế hệ sau.

10. Yếu tố nào sau đây là một ưu điểm của thuật toán `RuleBasedOptimizer` so với `GeneticAlgorithmOptimiz`

    (*a*) Luôn tìm ra giải pháp có tổng chi phí vận chuyển thấp nhất.

    (*b*) Tốc độ thực thi rất nhanh và logic đơn giản, dễ hiểu.

    (*c*) Có khả năng xử lý các bài toán có nhiều mục tiêu cùng lúc.

    (*d*) Đảm bảo đáp ứng 100% nhu cầu của tất cả các cửa hàng.

VI. Tài liệu tham khảo
Phụ lục

1. **Code:** Các file code được đề cập trong bài có thể được tải tại đây.
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng 3-4 tiếng.



Hình 11: Hình ảnh group facebook AIO Q&A.